

MODULE *CaIncarnationCore*

Incarnation-token *CA* store core — spec: 2026-06-10-ca-incarnation-store-design.md §12 + Appendix A. One key per hash; tokens = naturals per key; deletes exact-token via in-flight messages that may land arbitrarily late; *CAS* root manifests with embedded journal; *GC* = fold → retire → fence → recheck → delete. Sabotage* flags break one load-bearing rule each and MUST yield counterexamples. 2026-06-12 amendments (*B91* refresh): *EnableRegistry* models the namespace registry + manifest creation (W-REGISTER, *R3* step 0, fence-time universe + absent-shard minting); *EnableEvStale* models evidence staleness (amended W-EVIDENCE: stale tokenless evidence must be re-observed) plus dep-set tree-child validation (W-TREE-BUILD without a global presence oracle).

EXTENDS *Integers, Sequences, FiniteSets*

CONSTANTS

<i>Writers</i> ,	<i>e.g.</i> { <i>w1</i> , <i>w2</i> }
<i>Leaders</i> ,	<i>e.g.</i> { <i>L1</i> }; { <i>L1</i> , <i>L2</i> } only in stage 5
<i>Shards</i> ,	<i>e.g.</i> { <i>s1</i> }; { <i>s1</i> , <i>s2</i> } from stage 4
<i>Hashes</i> ,	content identities, <i>e.g.</i> { <i>h1</i> , <i>h2</i> }
<i>TreeHashes</i> ,	trees, subset of <i>Hashes</i> ({ } until stage 3)
<i>MaxToken</i> ,	token allocator bound per key
<i>MaxRound</i> ,	<i>GC</i> round bound
<i>MaxLog</i> ,	journal length bound per shard (state constraint)
<i>EnableResurrect</i> , <i>EnableTrees</i> , <i>EnableDebris</i> , <i>EnableSplit</i> , <i>EnableOverwrite</i> ,	
<i>EnableReval</i> ,	
<i>EnableRegistry</i> ,	2026-06-12 amendment: namespace registry + manifest creation (W-REGISTER, <i>R3</i> step 0)
<i>EnableEvStale</i> ,	2026-06-12 amendment: evidence staleness — re-observe stale tokenless evidence (W-EVIDENCE)
<i>SabotageNoFence</i> , <i>SabotageNoRecheckFold</i> , <i>SabotageNoRetireView</i> ,	
<i>SabotageUncondDelete</i> , <i>SabotageReusedTag</i> , <i>SabotageCascadeRace</i> , <i>SabotageCutOverclaim</i> ,	
<i>SabotageNoReobserve</i> ,	
<i>SabotageNoRegistry</i> ,	writers skip W-REGISTER: publish into unregistered namespaces, no floor — must dangle
<i>SabotageFoldTimeUniverse</i> ,	<i>GC</i> fences the FOLD-TIME universe (the <i>C++</i> hole fixed 2026-06-12) — must dangle
<i>SabotageNoEvReobserve</i>	gate admits STALE evidence without re-observation (Task-9 dangle) — must dangle

ASSUME *TreeHashes* ⊆ *Hashes*

NonTree ≜ *Hashes* \ *TreeHashes*

Children is DERIVED (*TLC* configs cannot express function/tuple constants). Asymmetric by design:

one tree (*FullTree*) references every non-tree hash; any further tree references exactly one.

With *TreeHashes* = {*t1*}: *t1* → *NonTree* (stages where one tree suffices).

With *TreeHashes* = {*t1*, *t2*}: full tree + single-child tree — covers BOTH shared-child survival (a child outliving one of two referencing trees) AND selective cascade (deleting the full tree must not touch the other tree's child). Nested subtrees are NOT modeled (one-level closure).

FullTree/Children are evaluated lazily by *TLC* — only ever applied to members of *TreeHashes*, so the CHOOSE is never evaluated when *TreeHashes* = { } (stages 1, 2, 5_{small}).

FullTree ≜ CHOOSE *t* ∈ *TreeHashes* : TRUE

OneChild ≜ CHOOSE *h* ∈ *NonTree* : TRUE

Children ≜ [*t* ∈ *TreeHashes* ↦ IF *t* = *FullTree* THEN *NonTree* ELSE { *OneChild* }]

Ev ≜ *MaxToken* + 1 dependency form: tokenless live-root evidence

$Toks \triangleq 1 \dots MaxToken$

$AddRec \triangleq [op : \{ \text{"add"}, \text{"rem"} \}, h : Hashes]$

$FenceRec \triangleq [op : \{ \text{"fence"} \}]$

$Rec \triangleq AddRec \cup FenceRec$

VARIABLES

—— S3 durable: objects ——

present, $[Hashes \rightarrow \text{BOOLEAN}]$
tokOf, $[Hashes \rightarrow 0 \dots MaxToken]$ current incarnation token (0 = never created)
nextTok, $[Hashes \rightarrow 1 \dots MaxToken + 1]$ fresh-token allocator (W-FRESH-TAG by construction)
deadTok, $[Hashes \rightarrow \text{SUBSET } Toks]$ tokens that have stopped being current (history; *INV_NO_RETURN*).
 MODEL RULE: any action that makes a token stop being current — in-place overwrite (*WResurrect*, and the future *WOverwrite* in stage 5) OR physical delete (Land) — MUST add that token here.
CondemnedAtView consults *deadTok*, so this is what rejects a stale token-bearing dependency.
 Omitting the *deadTok* update in any such action silently reintroduces a dangling-ref bug.
 NAMING: “dead as a publish dependency” (displaced by resurrect/overwrite OR deleted),
 NOT “physically deleted”. The *Apalache* proof core names this *obsoleteTok* for clarity.

—— S3 durable: roots + GC ——

man, $[Shards \rightarrow [\text{fence}: 0 \dots MaxRound, \text{refs}: \text{SUBSET } Hashes, \text{log}: Seq(Rec)]]$
retired, $\text{SUBSET } [h: Hashes, t: Toks, r: 1 \dots MaxRound]$
inflight, $\text{SUBSET } [h: Hashes, t: Toks]$ delete messages in flight; land ANY time later
gcRound, $0 \dots MaxRound$
gcPhase, $[Leaders \rightarrow \{ \text{"idle"}, \text{"retiring"}, \text{"fencing"}, \text{"fenced"} \}]$
roundOf, $[Leaders \rightarrow 0 \dots MaxRound]$
fencedSet, $[Leaders \rightarrow \text{SUBSET } Shards]$ shards this leader fenced this round
fencePos, $[Shards \rightarrow Nat]$ log position of the latest fence record
cursor, $[Shards \rightarrow Nat]$ folded prefix length (monotone)
trimBase, $[Shards \rightarrow Nat]$ journal trimmed below this (*INV_JOURNAL_COVERAGE*)
rootEdges, $\text{SUBSET } (Shards \times Hashes)$ snap: folded root edges
treeEdges, $\text{SUBSET } (TreeHashes \times Hashes)$ snap: expanded tree edges
marker, $\text{SUBSET } TreeHashes$ expansion markers ($marker(T) \equiv T$'s edges present)
everEdged, $\text{SUBSET } Hashes$ journal-known
pendCasc, $\text{SUBSET } TreeHashes$ deferred cascades (*SabotageCascadeRace* only)

—— writer local ——

wDeps, $[Writers \rightarrow \text{SUBSET } (Hashes \times (1 \dots MaxToken + 1))] (h, tok) \text{ or } (h, Ev)$
wView, $[Writers \rightarrow 0 \dots MaxRound]$ highest retire round refreshed

—— stage 4: heartbeats + debris + full-GC cut ——

creator, $[Hashes \rightarrow Writers \cup \{ \text{"none"} \}]$ who last (re)created the object
hbAlive, $[Writers \rightarrow \text{BOOLEAN}]$ heartbeat object present
hbSeq, $[Writers \rightarrow Nat]$ monotone renewal counter
wedged, $[Writers \rightarrow \text{BOOLEAN}]$ renewal thread stuck (writer still *acts!*)
hbObs, $[Writers \rightarrow Nat \cup \{ -1 \}]$ GC's first observation ($-1 = none$ yet)
fgPhase, $\{ \text{"idle"}, \text{"reading"}, \text{"read"} \}$ full-GC walk program counter
fgCut, $[Shards \rightarrow Nat]$ log length actually read per shard

$fgRefs,$ $[Shards \rightarrow \text{SUBSET } Hashes]$ $refs$ snapshot actually read per shard
 $fgSeen,$ $\text{SUBSET } Shards$ shards read so far this walk
 — 2026 – 06 – 12 amendments: namespace registry + evidence staleness —
 $reg,$ registry bundle (*EnableRegistry*; one model shard = one namespace):
 $.ns : \text{SUBSET } Shards$ registered namespaces (CAS-append-only)
 $.man : [Shards \rightarrow \text{BOOLEAN}]$ shard manifest EXISTS (created by publish or fence mint)
 $.fence : 0 \dots MaxRound$ registry *fence_round* (GC-written, monotone)
 $.floor : [Shards \rightarrow 0 \dots MaxRound]$ registry fence observed AT REGISTRATION — the
 weakest publish-gate floor any process is guaranteed to hold (the
 implementation’s *ensureRegistered* cache is monotone from this read)
 $.univ : [Leaders \rightarrow \text{SUBSET } Shards]$ this round’s shard-fence universe — captured at
 the REGISTRY FENCE (fence-time; the registry is append-only, so this is a
 superset of any earlier read). *SabotageFoldTimeUniverse* captures it at
 GStartRound instead — the *C++* hole fixed 2026 – 06 – 12.
 $.done : [Leaders \rightarrow \text{BOOLEAN}]$ registry fenced this round (orders *R3* step 0 first)
 wEv $[Writers \rightarrow \text{SUBSET } (Hashes \times 0 \dots MaxRound)]$ (*EnableEvStale*): tokenless evidence
 tagged with the view round it was RECORDED at — the staleness input of the amended
 W-EVIDENCE. Replaces the $wDeps\langle h, Ev \rangle$ form when the flag is on.

$vars \triangleq \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf,$
 $fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged,$
 $pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut,$
 $fgRefs, fgSeen, reg, wEv \rangle$

— helpers

A token-bearing dependency is condemned in a view if a retire entry for that exact token is
 visible at the view, OR the token is already physically dead (*INV_NO_RETURN*: that exact token
 can never again be a valid dependency, even after the retire entry was consumed by its landing).

$RetiredHit(h, t, v) \triangleq \exists e \in retired : e.h = h \wedge e.t = t \wedge e.r \leq v$
 $CondemnedAtView(h, t, v) \triangleq RetiredHit(h, t, v) \vee t \in deadTok[h]$
 $HashHitAtView(h, v) \triangleq \exists e \in retired : e.h = h \wedge e.r \leq v$
 $InDeg(h) \triangleq Cardinality(\{e \in rootEdges : e[2] = h\}) + Cardinality(\{e \in treeEdges : e[2] = h\})$
 $Max2(a, b) \triangleq \text{IF } a > b \text{ THEN } a \text{ ELSE } b$

Publish-gate fence floor (W-PUBLISH-GATE joined with W-REGISTER): the manifest’s *fence_round*
 (0 while the manifest does not exist) joined with the namespace’s REGISTRATION-TIME registry
 floor. With *EnableRegistry* = FALSE, $reg.floor[s] = 0$ and this reduces to $man[s].fence$.

$PubFloor(s) \triangleq Max2(man[s].fence, reg.floor[s])$

Amended W-EVIDENCE (2026 – 06 – 12): tokenless evidence recorded at view r is admissible iff the
 hash has no retire-entry hit at the writer’s view (a hit forces escalation — *WEvObserve* / a
 resurrect path) AND the evidence is FRESH ($r \geq wView$). Once retired entries drop on confirmed
 outcomes (*EnableReval*), “no hit” can mean “already deleted”, so STALE evidence (recorded under
 an older view) must be re-observed before it can gate a publish — exactly the Task – 9 fix.

SabotageNoEvReobserve admits stale evidence: the dangle MUST return.

$EvOK(w) \triangleq \forall d \in wEv[w] :$
 $\quad \wedge \neg HashHitAtView(d[1], wView[w])$

$$\begin{aligned}
& \wedge (SabotageNoEvReobserve \vee d[2] \geq wView[w]) \\
Reach(r) & \triangleq \{r\} \cup (\text{IF } r \in TreeHashes \text{ THEN } Children[r] \text{ ELSE } \{\}) \\
ReachableSet & \triangleq \text{UNION } \{Reach(r) : r \in \text{UNION } \{man[s].refs : s \in Shards\}\} \\
FoldedThroughFence & \triangleq \forall s \in Shards : cursor[s] \geq fencePos[s] \\
Init & \triangleq \\
& \wedge present = [h \in Hashes \mapsto \text{FALSE}] \\
& \wedge tokOf = [h \in Hashes \mapsto 0] \\
& \wedge nextTok = [h \in Hashes \mapsto 1] \\
& \wedge deadTok = [h \in Hashes \mapsto \{\}] \\
& \wedge man = [s \in Shards \mapsto [fence \mapsto 0, refs \mapsto \{\}, log \mapsto \langle \rangle]] \\
& \wedge retired = \{\} \wedge inflight = \{\} \wedge gcRound = 0 \\
& \wedge gcPhase = [l \in Leaders \mapsto \text{"idle"}] \\
& \wedge roundOf = [l \in Leaders \mapsto 0] \\
& \wedge fencedSet = [l \in Leaders \mapsto \{\}] \\
& \wedge fencePos = [s \in Shards \mapsto 0] \\
& \wedge cursor = [s \in Shards \mapsto 0] \\
& \wedge trimBase = [s \in Shards \mapsto 0] \\
& \wedge rootEdges = \{\} \wedge treeEdges = \{\} \wedge marker = \{\} \wedge everEdged = \{\} \wedge pendCasc = \{\} \\
& \wedge wDeps = [w \in Writers \mapsto \{\}] \\
& \wedge wView = [w \in Writers \mapsto 0] \\
& \wedge creator = [h \in Hashes \mapsto \text{"none"}] \\
& \wedge hbAlive = [w \in Writers \mapsto \text{FALSE}] \\
& \wedge hbSeq = [w \in Writers \mapsto 0] \\
& \wedge wedged = [w \in Writers \mapsto \text{FALSE}] \\
& \wedge hbObs = [w \in Writers \mapsto -1] \\
& \wedge fgPhase = \text{"idle"} \\
& \wedge fgCut = [s \in Shards \mapsto 0] \\
& \wedge fgRefs = [s \in Shards \mapsto \{\}] \\
& \wedge fgSeen = \{\}
\end{aligned}$$

With *EnableRegistry* off, the registry is pre-populated and inert: every namespace registered, every manifest existing, all floors 0 — old stage behavior is unchanged.

$$\begin{aligned}
& \wedge reg = \text{IF } EnableRegistry \\
& \quad \text{THEN } [ns \mapsto \{\}, man \mapsto [s \in Shards \mapsto \text{FALSE}], fence \mapsto 0, \\
& \quad \quad floor \mapsto [s \in Shards \mapsto 0], univ \mapsto [l \in Leaders \mapsto \{\}], \\
& \quad \quad done \mapsto [l \in Leaders \mapsto \text{FALSE}]] \\
& \quad \text{ELSE } [ns \mapsto Shards, man \mapsto [s \in Shards \mapsto \text{TRUE}], fence \mapsto 0, \\
& \quad \quad floor \mapsto [s \in Shards \mapsto 0], univ \mapsto [l \in Leaders \mapsto Shards], \\
& \quad \quad done \mapsto [l \in Leaders \mapsto \text{TRUE}]] \\
& \wedge wEv = [w \in Writers \mapsto \{\}]
\end{aligned}$$

— writer actions

Create a missing object: fresh token from the allocator (W-SAME-CONTENT is by construction: the model's key IS the content). Dependency recorded with the created token.

$$WCreate(w, h) \triangleq$$

$\wedge$ $present[h] = \text{FALSE} \wedge nextTok[h] \leq MaxToken$
 $\wedge$ $(h \in TreeHashes \Rightarrow \forall c \in Children[h] : present[c])$
 $\wedge$ $(\neg EnableDebris) \vee hbAlive[w]$ W-HEARTBEAT: durable before the first object PUT
 $\wedge$ $present' = [present \text{ EXCEPT } ![h] = \text{TRUE}]$
 $\wedge$ $tokOf' = [tokOf \text{ EXCEPT } ![h] = nextTok[h]]$
 $\wedge$ $nextTok' = [nextTok \text{ EXCEPT } ![h] = @ + 1]$
 $\wedge$ $creator' = \text{IF } EnableDebris \text{ THEN } [creator \text{ EXCEPT } ![h] = w] \text{ ELSE } creator$
 $\wedge$ $wDeps' = [wDeps \text{ EXCEPT } ![w] = @ \cup \{ \langle h, nextTok[h] \rangle \}]$
 $\wedge$ $\text{UNCHANGED } \langle deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet,$
 $\quad fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged,$
 $\quad pendCasc, wView, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs,$
 $\quad fgSeen, reg, wEv \rangle$

Cold reuse: the existence check observes the current token (token-bearing dependency).

$WReuse(w, h) \triangleq$
 $\wedge present[h]$
 $\wedge wDeps' = [wDeps \text{ EXCEPT } ![w] = @ \cup \{ \langle h, tokOf[h] \rangle \}]$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase,$
 $\quad roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker,$
 $\quad everEdged, pendCasc, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase,$
 $\quad fgCut, fgRefs, fgSeen, reg, wEv \rangle$

Anonymous environment churn: an unconditional same-content re-PUT by anyone, any time (spec §12: safety must not rest on PUT conditions). No dependency recorded — writer-intent overwrites are $WResurrect$ (condemned) / $WReuse$ (adopt). Per the $deadTok$ MODEL RULE, the in-place overwrite makes the OLD token stop being current, so it MUST join $deadTok$ (else a stale token-bearing dep on it dangles — the bug $WResurrect$ fixed). creator gated on $EnableDebris$ (keeps earlier stages inert).

$WOverwrite(w, h) \triangleq$
 $\wedge EnableOverwrite \wedge present[h] \wedge nextTok[h] \leq MaxToken$
 $\wedge tokOf' = [tokOf \text{ EXCEPT } ![h] = nextTok[h]]$
 $\wedge nextTok' = [nextTok \text{ EXCEPT } ![h] = @ + 1]$
 $\wedge deadTok' = [deadTok \text{ EXCEPT } ![h] = @ \cup \{ tokOf[h] \}]$
 $\wedge creator' = \text{IF } EnableDebris \text{ THEN } [creator \text{ EXCEPT } ![h] = w] \text{ ELSE } creator$
 $\wedge \text{UNCHANGED } \langle present, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet,$
 $\quad fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc,$
 $\quad wDeps, wView, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

Resurrect a condemned current incarnation: overwrite in place with a FRESH token (W-FRESH-TAG).

The overwrite is an in-place replacement of the single physical slot, so the OLD token is gone for good — record it in $deadTok$ exactly as a physical delete would (INV-NO-RETURN: the overwritten token can never again be a valid dependency, even though its retire entry is later consumed by a landing or dropped as spared at the recheck).

Without this, a writer holding a stale token-bearing dependency on the overwritten incarnation could pass the publish gate after that incarnation's retire entry drops, then a delete of the NEW token leaves the logical hash absent under a live root (INV_NO_DANGLE / INV_NO_LOSS).

$SabotageReusedTag$ re-issues the condemned token itself — token recurrence, must break INV_NO_RETURN.

$$\begin{aligned}
WResurrect(w, h) &\triangleq \\
&\wedge EnableResurrect \\
&\wedge present[h] \wedge CondemnedAtView(h, tokOf[h], wView[w]) \\
&\wedge nextTok[h] \leq MaxToken \\
&\wedge (\neg EnableDebris) \vee hbAlive[w] \quad \text{W-HEARTBEAT: durable before the first object PUT} \\
&\wedge LET newt \triangleq IF SabotageReusedTag THEN tokOf[h] ELSE nextTok[h] IN \\
&\quad \wedge tokOf' = [tokOf \text{ EXCEPT } ![h] = newt] \\
&\quad \wedge nextTok' = [nextTok \text{ EXCEPT } ![h] = @ + 1] \\
&\quad \wedge deadTok' = [deadTok \text{ EXCEPT } ![h] = @ \cup \{tokOf[h]\}] \\
&\quad \wedge wDeps' = [wDeps \text{ EXCEPT } ![w] = @ \cup \{\langle h, newt \rangle\}] \\
&\wedge creator' = IF EnableDebris THEN [creator \text{ EXCEPT } ![h] = w] ELSE creator \\
&\wedge UNCHANGED \langle present, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, \\
&\quad fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, \\
&\quad wView, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle
\end{aligned}$$

Carry-forward / fetch-by-reference: tokenless live-root-evidence dependency (no request made). Under *EnableEvStale* the evidence is RECORDED WITH the writer's current view round (the staleness input of the amended W-EVIDENCE); otherwise the legacy *wDeps* $\langle h, Ev \rangle$ form. LIVE-ROOT is literal under *EnableEvStale*: the object must be REACHABLE when observed (the writer carries it forward from an existing live tree/ref — W-EVIDENCE's definition). Found by TLC: present-only evidence on an already-unreachable object can be condemned and deleted in the same round it was recorded, staying “fresh” past the gate — a dangle. (The legacy mode keeps the liberal present-only guard: there the global-present *TreeDepsOK* covers children, and more behaviors = stronger coverage of the old stages.)

$$\begin{aligned}
WEvidence(w, h) &\triangleq \\
&\wedge EnableResurrect \wedge present[h] \\
&\wedge (\neg EnableEvStale) \vee h \in ReachableSet \\
&\wedge IF EnableEvStale \\
&\quad THEN \wedge wEv' = [wEv \text{ EXCEPT } ![w] = @ \cup \{\langle h, wView[w] \rangle\}] \\
&\quad \wedge wDeps' = wDeps \\
&\quad ELSE \wedge wDeps' = [wDeps \text{ EXCEPT } ![w] = @ \cup \{\langle h, Ev \rangle\}] \\
&\quad \wedge wEv' = wEv \\
&\wedge UNCHANGED \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, \\
&\quad roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, \\
&\quad everEdged, pendCasc, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, \\
&\quad fgCut, fgRefs, fgSeen, reg \rangle
\end{aligned}$$

W-EVIDENCE escalation: a retire-view hit on the hash forces resolution to a token-bearing entry (adopt the current token iff it is not condemned; a condemned current token goes via *WResurrect*).

$$\begin{aligned}
WResolveEvidence(w, h) &\triangleq \\
&\wedge EnableResurrect \\
&\wedge \langle h, Ev \rangle \in wDeps[w] \wedge present[h] \\
&\wedge \neg CondemnedAtView(h, tokOf[h], wView[w]) \\
&\wedge wDeps' = [wDeps \text{ EXCEPT } ![w] = (@ \setminus \{\langle h, Ev \rangle\}) \cup \{\langle h, tokOf[h] \rangle\}] \\
&\wedge UNCHANGED \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase,
\end{aligned}$$

roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv

Amended W-EVIDENCE re-observation: ONE HEAD on an evidence hash — present resolves the evidence to the CURRENT token (a token-bearing dependency, re-validated by *DepOK* at publish); absent drops it (the writer re-creates via *WCreate* or abandons). Allowed any time — re-observation is always sound; *STALENESS* matters only in the gate (*EvOK*), which is what forces this step.

$WEvObserve(w, h) \triangleq$
 $\wedge EnableEvStale$
 $\wedge \exists r \in 0 \dots MaxRound : \langle h, r \rangle \in wEv[w]$
 $\wedge wEv' = [wEv \text{ EXCEPT } ![w] = \{d \in @ : d[1] \neq h\}]$
 $\wedge wDeps' = \text{IF } present[h] \text{ THEN } [wDeps \text{ EXCEPT } ![w] = @ \cup \{\langle h, tokOf[h] \rangle\}] \text{ ELSE } wDeps$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg \rangle$

W-REGISTER (spec §5 amendment 2026 – 06 – 12): a namespace is CAS-appended to the registry before its first publish; the registering read-modify-write observes the CURRENT registry *fence_round*, which becomes the namespace's publish-gate floor (monotone lower bound for every later process).

$WRegister(s) \triangleq$
 $\wedge EnableRegistry \wedge s \notin reg.ns$
 $\wedge reg' = [reg \text{ EXCEPT } !.ns = @ \cup \{s\},$
 $\quad \quad \quad !.floor = [@ \text{ EXCEPT } ![s] = reg.fence]]$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, wEv \rangle$

W-REVALIDATE HEAD (*EnableReval*): on a retire-view refresh the writer re-observes every token-bearing dependency on *h* — refresh to the CURRENT token if the key is present, or drop the dependency if the key is absent (re-create handled elsewhere). *Ev* members are untouched.

$WReobserve(w, h) \triangleq$
 $\wedge EnableReval$
 $\wedge \exists t \in Toks : \langle h, t \rangle \in wDeps[w]$
 $\wedge \text{LET } keep \triangleq \{d \in wDeps[w] : d[1] \neq h \vee d[2] = Ev\}$
 $\quad \text{IN } wDeps' = [wDeps \text{ EXCEPT } ![w] = \text{IF } present[h] \text{ THEN } keep \cup \{\langle h, tokOf[h] \rangle\} \text{ ELSE } keep]$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

The round a retire-view refresh can CLAIM COVERAGE of. In the implementation *gc/state.round* advances only when a round's retire sets are durable (*R2*'s *CAS*), so a view “at round *R*” sees ALL round-*R* entries. Model *gcRound* bumps at *GStartRound* — while the *gcRound*-runner is still

RETIRING, a refresh can claim only $gcRound - 1$. (Found by *TLC*: evidence freshness $r \geq wView$ is sound ONLY with this coverage property | an “early” round $- R$ view recorded into evidence let a later same $-$ round retire slip past the freshness check and dangle a tree child.)

$ViewableRound \triangleq$ IF $\exists l \in Leaders : gcPhase[l] = \text{“retiring”} \wedge roundOf[l] = gcRound$
 THEN $gcRound - 1$ ELSE $gcRound$

$WRefreshView(w) \triangleq$

$\wedge wView' = [wView \text{ EXCEPT } ![w] = ViewableRound]$

$\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wDeps, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

Publish : one atomic successful CAS | guard reads the CURRENT manifest (CAS linearization).

$W - PUBLISH - GATE + W - EVIDENCE$. SabotageNoRetireView removes the gate.

$W - REVALIDATE \text{ mode}(EnableReval)$: the gate has NO dead $-$ token oracle | entries drop on outcomes, so a stale observation is validated by $RE - OBSERVATION$: the dep s token must be the CURRENT one ($present \wedge tokOf = t$), atomically with the publish CAS (the fence closes the $HEAD \rightarrow CAS$ window on the GC side; overwrites without a fence are harmless — the ref names the hash).

SabotageNoReobserve drops the re-observation conjunct: the gate degrades to retired-only and the stage $- 1$ stale-dependency dangle MUST return (the $F1$ negative control).

$DepOK(w) \triangleq$

$\forall d \in wDeps[w] :$

IF $d[2] = Ev$ THEN $\neg HashHitAtView(d[1], wView[w])$

ELSE IF *EnableReval*

THEN $\wedge \neg RetiredHit(d[1], d[2], wView[w])$

$\wedge SabotageNoReobserve \vee (present[d[1]] \wedge tokOf[d[1]] = d[2])$

ELSE $\neg CondemnedAtView(d[1], d[2], wView[w])$

A published tree’s children must be present AND live (not condemned in the writer’s view) at publish time. Models the bottom-up build (children uploaded before the tree ref is published).

This models the CHILD half of the dependency-set gate as a publish-time LIVE re-read of each child’s current token (it does NOT carry separate per-child $wDeps$ entries) — sound because all incarnations are payload-identical (W-SAME-CONTENT) and trees reference children by hash, so validating the current token matches the reader contract. Without this a writer can publish a tree over an absent/condemned child and dangle (*INV_NO_LOSS*). One-level: stage $- 3$ trees are flat ($Children[t]$ subset of *NonTree*); nested subtrees are a documented residual.

Under *EnableEvStale* the global-presence read goes away: children are validated through the writer’s OWN dependency set (a token dep checked by *DepOK*, or evidence checked by *EvOK*, at this same publish) — faithful to W-TREE-BUILD, where the implementation has no presence oracle and every child must be covered by a created/reused/evidence dependency.

$TreeDepsOK(w, h) \triangleq h \in TreeHashes \Rightarrow$

$\forall c \in Children[h] :$

IF *EnableEvStale*

THEN $\vee \exists t \in Toks : \langle c, t \rangle \in wDeps[w]$

$\vee \exists r \in 0 \dots MaxRound : \langle c, r \rangle \in wEv[w]$

ELSE $\wedge present[c]$

$$\begin{aligned}
& \wedge \text{IF } \textit{EnableReval} \text{ THEN } \neg \textit{RetiredHit}(c, \textit{tokOf}[c], \textit{wView}[w]) \\
& \quad \text{ELSE } \neg \textit{CondemnedAtView}(c, \textit{tokOf}[c], \textit{wView}[w]) \\
\textit{WPublish}(w, s, h) \triangleq & \\
& \wedge \exists t \in \textit{Toks} : \langle h, t \rangle \in \textit{wDeps}[w] \quad \text{the root itself was created/reused} \\
& \wedge (\textit{SabotageNoRegistry} \vee s \in \textit{reg.ns}) \quad \text{W-REGISTER: register before any publish} \\
& \wedge h \notin \textit{man}[s].\textit{refs} \\
& \wedge \textit{Len}(\textit{man}[s].\textit{log}) < \textit{MaxLog} \\
& \wedge \textit{SabotageNoRetireView} \vee (\textit{wView}[w] \geq \textit{PubFloor}(s) \wedge \textit{DepOK}(w) \wedge \textit{EvOK}(w)) \\
& \wedge \textit{TreeDepsOK}(w, h) \\
& \wedge \textit{man}' = [\textit{man} \text{ EXCEPT } ![s].\textit{refs} = @ \cup \{h\}, \\
& \quad \quad \quad ![s].\textit{log} = \textit{Append}(@, [\textit{op} \mapsto \text{"add"}, h \mapsto h])] \\
& \wedge \textit{reg}' = [\textit{reg} \text{ EXCEPT } !\textit{man}[s] = \text{TRUE}] \quad \text{first publish CREATES the manifest (no-op after)} \\
& \wedge \textit{wDeps}' = [\textit{wDeps} \text{ EXCEPT } ![w] = \{\}] \\
& \wedge \textit{wEv}' = [\textit{wEv} \text{ EXCEPT } ![w] = \{\}] \\
& \wedge \text{UNCHANGED } \langle \textit{present}, \textit{tokOf}, \textit{nextTok}, \textit{deadTok}, \textit{retired}, \textit{inflight}, \textit{gcRound}, \textit{gcPhase}, \textit{roundOf}, \\
& \quad \textit{fencedSet}, \textit{fencePos}, \textit{cursor}, \textit{trimBase}, \textit{rootEdges}, \textit{treeEdges}, \textit{marker}, \textit{everEdged}, \\
& \quad \textit{pendCasc}, \textit{wView}, \textit{creator}, \textit{hbAlive}, \textit{hbSeq}, \textit{wedged}, \textit{hbObs}, \textit{fgPhase}, \textit{fgCut}, \\
& \quad \textit{fgRefs}, \textit{fgSeen} \rangle \\
\textit{WDrop}(s, h) \triangleq & \\
& \wedge h \in \textit{man}[s].\textit{refs} \\
& \wedge \textit{Len}(\textit{man}[s].\textit{log}) < \textit{MaxLog} \\
& \wedge \textit{man}' = [\textit{man} \text{ EXCEPT } ![s].\textit{refs} = @ \setminus \{h\}, \\
& \quad \quad \quad ![s].\textit{log} = \textit{Append}(@, [\textit{op} \mapsto \text{"rem"}, h \mapsto h])] \\
& \wedge \text{UNCHANGED } \langle \textit{present}, \textit{tokOf}, \textit{nextTok}, \textit{deadTok}, \textit{retired}, \textit{inflight}, \textit{gcRound}, \textit{gcPhase}, \textit{roundOf}, \\
& \quad \textit{fencedSet}, \textit{fencePos}, \textit{cursor}, \textit{trimBase}, \textit{rootEdges}, \textit{treeEdges}, \textit{marker}, \textit{everEdged}, \\
& \quad \textit{pendCasc}, \textit{wDeps}, \textit{wView}, \textit{creator}, \textit{hbAlive}, \textit{hbSeq}, \textit{wedged}, \textit{hbObs}, \textit{fgPhase}, \\
& \quad \textit{fgCut}, \textit{fgRefs}, \textit{fgSeen}, \textit{reg}, \textit{wEv} \rangle \\
\textit{WAbandon}(w) \triangleq & \quad \text{crash/abort before publish: deps lost; uploads remain (debris in stage 4)} \\
& \wedge (\textit{wDeps}[w] \neq \{\} \vee \textit{wEv}[w] \neq \{\}) \\
& \wedge \textit{wDeps}' = [\textit{wDeps} \text{ EXCEPT } ![w] = \{\}] \\
& \wedge \textit{wEv}' = [\textit{wEv} \text{ EXCEPT } ![w] = \{\}] \\
& \wedge \text{UNCHANGED } \langle \textit{present}, \textit{tokOf}, \textit{nextTok}, \textit{deadTok}, \textit{man}, \textit{retired}, \textit{inflight}, \textit{gcRound}, \textit{gcPhase}, \\
& \quad \textit{roundOf}, \textit{fencedSet}, \textit{fencePos}, \textit{cursor}, \textit{trimBase}, \textit{rootEdges}, \textit{treeEdges}, \textit{marker}, \\
& \quad \textit{everEdged}, \textit{pendCasc}, \textit{wView}, \textit{creator}, \textit{hbAlive}, \textit{hbSeq}, \textit{wedged}, \textit{hbObs}, \textit{fgPhase}, \\
& \quad \textit{fgCut}, \textit{fgRefs}, \textit{fgSeen}, \textit{reg} \rangle
\end{aligned}$$

GC actions (per leader)

$$\begin{aligned}
\textit{GStartRound}(l) \triangleq & \\
& \wedge \textit{gcPhase}[l] = \text{"idle"} \wedge \textit{gcRound} < \textit{MaxRound} \\
& \wedge \textit{gcRound}' = \textit{gcRound} + 1 \\
& \wedge \textit{roundOf}' = [\textit{roundOf} \text{ EXCEPT } ![l] = \textit{gcRound} + 1] \\
& \wedge \textit{gcPhase}' = [\textit{gcPhase} \text{ EXCEPT } ![l] = \text{"retiring"}] \\
& \wedge \textit{fencedSet}' = [\textit{fencedSet} \text{ EXCEPT } ![l] = \{\}] \\
& \text{The registry fence is pending for this round. Under } \textit{SabotageFoldTimeUniverse} \text{ the}
\end{aligned}$$

shard-fence universe is captured HERE (the fold-time registry read) — the *C++* hole:
a namespace registered between now and *GFenceRegistry* is never fenced this round.

```

 $\wedge \text{reg}' = \text{IF } \text{EnableRegistry}$ 
    THEN  $[\text{reg} \text{ EXCEPT } !.\text{univ} = [\text{@} \text{ EXCEPT } ![l] = \text{IF } \text{SabotageFoldTimeUniverse}$ 
        THEN  $\text{reg}.\text{ns} \text{ ELSE } \{\}$ ],
         $!.done = [\text{@} \text{ EXCEPT } ![l] = \text{FALSE}]$ 
    ELSE  $\text{reg}$ 
 $\wedge \text{UNCHANGED } \langle \text{present}, \text{tokOf}, \text{nextTok}, \text{deadTok}, \text{man}, \text{retired}, \text{inflight}, \text{fencePos}, \text{cursor},$ 
     $\text{trimBase}, \text{rootEdges}, \text{treeEdges}, \text{marker}, \text{everEdged}, \text{pendCasc}, \text{wDeps}, \text{wView},$ 
     $\text{creator}, \text{hbAlive}, \text{hbSeq}, \text{wedged}, \text{hbObs}, \text{fgPhase}, \text{fgCut}, \text{fgRefs}, \text{fgSeen}, \text{wEv} \rangle$ 

```

Fold one journal record into the snap (edge-set semantics; expansion marker rule).

```

 $\text{GFold}(s) \triangleq$ 
 $\wedge \text{cursor}[s] < \text{Len}(\text{man}[s].\text{log})$ 
 $\wedge \text{LET } \text{rec} \triangleq \text{man}[s].\text{log}[\text{cursor}[s] + 1] \text{IN}$ 
     $\wedge \text{rootEdges}' = \text{CASE } \text{rec.op} = \text{"add"} \rightarrow \text{rootEdges} \cup \{\langle s, \text{rec.h} \rangle\}$ 
         $\square \text{rec.op} = \text{"rem"} \rightarrow \text{rootEdges} \setminus \{\langle s, \text{rec.h} \rangle\}$ 
         $\square \text{rec.op} = \text{"fence"} \rightarrow \text{rootEdges}$ 
     $\wedge \text{IF } \text{rec.op} = \text{"add"} \wedge \text{rec.h} \in \text{TreeHashes} \wedge \text{rec.h} \notin \text{marker}$ 
        THEN  $\wedge \text{treeEdges}' = \text{treeEdges} \cup \{\langle \text{rec.h}, c \rangle : c \in \text{Children}[\text{rec.h}]\}$ 
             $\wedge \text{marker}' = \text{marker} \cup \{\text{rec.h}\}$ 
             $\wedge \text{everEdged}' = \text{everEdged} \cup \{\text{rec.h}\} \cup \text{Children}[\text{rec.h}]$ 
        ELSE  $\wedge \text{treeEdges}' = \text{treeEdges} \wedge \text{marker}' = \text{marker}$ 
             $\wedge \text{everEdged}' = \text{IF } \text{rec.op} = \text{"add"} \text{ THEN } \text{everEdged} \cup \{\text{rec.h}\} \text{ ELSE } \text{everEdged}$ 
 $\wedge \text{cursor}' = [\text{cursor} \text{ EXCEPT } ![s] = \text{@} + 1]$ 
 $\wedge \text{UNCHANGED } \langle \text{present}, \text{tokOf}, \text{nextTok}, \text{deadTok}, \text{man}, \text{retired}, \text{inflight}, \text{gcRound}, \text{gcPhase},$ 
     $\text{roundOf}, \text{fencedSet}, \text{fencePos}, \text{trimBase}, \text{pendCasc}, \text{wDeps}, \text{wView}, \text{creator},$ 
     $\text{hbAlive}, \text{hbSeq}, \text{wedged}, \text{hbObs}, \text{fgPhase}, \text{fgCut}, \text{fgRefs}, \text{fgSeen}, \text{reg}, \text{wEv} \rangle$ 

```

Retire a journal-known, present, in-degree – 0 candidate at its CURRENT token (the HEAD).

```

 $\text{GRetire}(l, h) \triangleq$ 
 $\wedge \text{gcPhase}[l] = \text{"retiring"}$ 
 $\wedge \text{present}[h] \wedge h \in \text{everEdged} \wedge \text{InDeg}(h) = 0$ 
 $\wedge \neg \exists e \in \text{retired} : e.h = h \wedge e.t = \text{tokOf}[h]$ 
 $\wedge \text{retired}' = \text{retired} \cup \{[h \mapsto h, t \mapsto \text{tokOf}[h], r \mapsto \text{roundOf}[l]]\}$ 
 $\wedge \text{UNCHANGED } \langle \text{present}, \text{tokOf}, \text{nextTok}, \text{deadTok}, \text{man}, \text{inflight}, \text{gcRound}, \text{gcPhase}, \text{roundOf},$ 
     $\text{fencedSet}, \text{fencePos}, \text{cursor}, \text{trimBase}, \text{rootEdges}, \text{treeEdges}, \text{marker}, \text{everEdged},$ 
     $\text{pendCasc}, \text{wDeps}, \text{wView}, \text{creator}, \text{hbAlive}, \text{hbSeq}, \text{wedged}, \text{hbObs}, \text{fgPhase},$ 
     $\text{fgCut}, \text{fgRefs}, \text{fgSeen}, \text{reg}, \text{wEv} \rangle$ 

```

R3 step 0 (amendment 2026 – 06 – 12): *CAS* the registry fence FIRST — the ordering point for namespace CREATION (a registration below this fence is in this round's universe; one after it observes $\text{fence} \geq \text{round}$ as its floor). The registry decoded by the COMMITTED fence *CAS* is ALSO the shard-fence universe — fence-time, a superset of any earlier read (the registry is append-only). *SabotageFoldTimeUniverse* keeps the universe captured at *GStartRound* instead: a namespace registered in between falls between the two horns — never fenced this round, yet its

registration floor predates this fence. That is the $C++$ hole fixed 2026-06-12.

$GFenceRegistry(l) \triangleq$
 $\wedge EnableRegistry$
 $\wedge gcPhase[l] = \text{"retiring"} \wedge \neg reg.done[l]$
 $\wedge LET\ univ \triangleq IF\ SabotageFoldTimeUniverse\ THEN\ reg.univ[l]\ ELSE\ reg.nsIN$
 $\wedge reg' = [reg\ EXCEPT\ !.fence = Max2(@, roundOf[l]),$
 $\quad !.univ = [@\ EXCEPT\ ![l] = univ],$
 $\quad !.done = [@\ EXCEPT\ ![l] = TRUE]]$
 The registry fence ENDS this round's retiring ($R2$ completes before $R3$ step 0 — the implementation's sequential round). Load-bearing: a writer registering AFTER this fence takes $floor = round$, and "view at round" must see ALL round entries — retires landing after the registry fence would be invisible to that view (a dangle, found by TLC when this transition was missing). An empty universe completes the fence phase right here.
 $\wedge gcPhase' = [gcPhase\ EXCEPT\ ![l] = IF\ univ = \{\}\ THEN\ \text{"fenced"}\ ELSE\ \text{"fencing"}]$
 $\wedge UNCHANGED\ \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, roundOf,$
 $\quad fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged,$
 $\quad pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase,$
 $\quad fgCut, fgRefs, fgSeen, wEv \rangle$

Fence one shard: bump $fence_round$ in the manifest (a CAS — appends a fence record). Under $EnableRegistry$ the universe is $reg.univ[l]$ (captured by $GFenceRegistry$, which must run first) and fencing an absent manifest MINTS it (the create-if-absent CAS ; the create race against a first publish is the required total order).

$SabotageNoFence$: the fence does NOT touch the manifest (writers never blocked, horn 2 open).

$GFenceShard(l, s) \triangleq$
 $\wedge gcPhase[l] \in \{\text{"retiring"}, \text{"fencing"}\}$
 $\wedge (\neg EnableRegistry) \vee reg.done[l]$ registry fence first ($R3$ step 0)
 $\wedge s \in reg.univ[l] \wedge s \notin fencedSet[l]$
 $\wedge Len(man[s].log) < MaxLog$
 $\wedge IF\ SabotageNoFence$
 $\quad THEN\ man' = man \wedge fencePos' = fencePos \wedge reg' = reg$
 $\quad fence\ is\ monotone\ (CAS\text{-modelled}): a\ stale\ leader's\ lower\ round\ is\ absorbed,\ never\ lowers\ it\ (INV\text{-}MONOTONE)$
 $\quad ELSE\ \wedge man' = [man\ EXCEPT\ ![s].fence = IF\ roundOf[l] > man[s].fence\ THEN\ roundOf[l]\ ELSE\ man[s].fence$
 $\quad \quad ![s].log = Append(@, [op \mapsto \text{"fence"}])]$
 $\quad \wedge fencePos' = [fencePos\ EXCEPT\ ![s] = Len(man[s].log) + 1]$
 $\quad \wedge reg' = [reg\ EXCEPT\ !.man[s] = TRUE]$ mint (no-op if the manifest exists)
 $\wedge fencedSet' = [fencedSet\ EXCEPT\ ![l] = @ \cup \{s\}]$
 $\wedge gcPhase' = [gcPhase\ EXCEPT\ ![l] = IF\ reg.univ[l] \subseteq (fencedSet[l] \cup \{s\})$
 $\quad THEN\ \text{"fenced"}\ ELSE\ \text{"fencing"}]$
 $\wedge UNCHANGED\ \langle present, tokOf, nextTok, deadTok, retired, inflight, gcRound, roundOf, cursor,$
 $\quad trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wDeps, wView,$
 $\quad creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, wEv \rangle$

$Recheck +$ issue delete. The recheck requires the fold to have provably reached every fence position ($SabotageNoRecheckFold$ drops that — horn 1 open). Spared entries drop at the recheck;

a condemned entry STAYS in *retired* (still blocking reuse) and a delete MESSAGE is sent |
the entry drops only when the message lands (spec 7 : entries drop on confirmed outcomes only).

$GRcheckDelete(l, e) \triangleq$
 $\wedge gcPhase[l] = \text{"fenced"} \wedge e \in retired \wedge e.r = roundOf[l]$
 $\wedge SabotageNoRecheckFold \vee FoldedThroughFence$
 $\wedge IF InDeg(e.h) > 0$
 THEN $\wedge retired' = retired \setminus \{e\}$ outcome = spared
 $\wedge inflight' = inflight$
 ELSE $\wedge [h \mapsto e.h, t \mapsto e.t] \notin inflight$ no duplicate sends (idempotent anyway)
 $\wedge retired' = retired$ entry kept until the landing confirms
 $\wedge inflight' = inflight \cup \{[h \mapsto e.h, t \mapsto e.t]\}$
 $\wedge UNCHANGED \langle present, tokOf, nextTok, deadTok, man, gcRound, gcPhase, roundOf, fencedSet,$
 $fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc,$
 $wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs,$
 $fgSeen, reg, wEv \rangle$

$GEndRound(l) \triangleq$
 $\wedge gcPhase[l] = \text{"fenced"}$
 $\wedge \neg \exists e \in retired : e.r = roundOf[l]$ waits for landings : entries drop only on outcomes
 $\wedge gcPhase' = [gcPhase \text{ EXCEPT } ![l] = \text{"idle"}]$
 $\wedge UNCHANGED \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, roundOf,$
 $fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged,$
 $pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase,$
 $fgCut, fgRefs, fgSeen, reg, wEv \rangle$

A delete message lands : exact – token(412 = no – op). SabotageUncondDelete ignores the token.
The landing is the confirmed outcome : the matching retired entry drops HERE (deleted/absent/
replaced), never at send time. Trees : cascade(strip child edges + marker) is ATOMIC with the
landing | the pipeline rule. SabotageCascadeRace defers the strip to an arbitrary later time.

$Land(d) \triangleq$
 $\wedge d \in inflight$
 $\wedge inflight' = inflight \setminus \{d\}$
 $\wedge retired' = \{e \in retired : \neg(e.h = d.h \wedge e.t = d.t)\}$
 $\wedge IF present[d.h] \wedge (SabotageUncondDelete \vee tokOf[d.h] = d.t)$
 THEN $\wedge present' = [present \text{ EXCEPT } ![d.h] = \text{FALSE}]$
 $\wedge deadTok' = [deadTok \text{ EXCEPT } ![d.h] = @ \cup \{tokOf[d.h]\}]$
 $\wedge IF d.h \in TreeHashes \wedge d.h \in marker$
 THEN IF SabotageCascadeRace
 THEN $\wedge pendCasc' = pendCasc \cup \{d.h\}$
 $\wedge treeEdges' = treeEdges \wedge marker' = marker$
 ELSE $\wedge treeEdges' = \{e \in treeEdges : e[1] \neq d.h\}$
 $\wedge marker' = marker \setminus \{d.h\}$
 $\wedge pendCasc' = pendCasc$
 ELSE UNCHANGED $\langle treeEdges, marker, pendCasc \rangle$
 ELSE $\wedge UNCHANGED \langle present, deadTok, treeEdges, marker, pendCasc \rangle$ 412/absent

$\wedge$ UNCHANGED $\langle tokOf, nextTok, man, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, everEdged, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

Deferred cascade application(exists ONLY under SabotageCascadeRace; this is the race).

$ApplyPendCascade(t) \triangleq$

$\wedge SabotageCascadeRace \wedge t \in pendCasc$

$\wedge treeEdges' = \{e \in treeEdges : e[1] \neq t\}$

$\wedge marker' = marker \setminus \{t\}$

$\wedge pendCasc' = pendCasc \setminus \{t\}$

$\wedge$ UNCHANGED $\langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, everEdged, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

P9(2026 – 06 – 17) : GC removes an absent, zero – in – degree, journal – known node from everEdged

Models BOTH implementation prune sites as one abstract operation | the cascade s delete-time

prune and the retire observe loop's HEAD – 404 prune both do exactly this. Always enabled when the

node is gone: a single unconditional action is the STRONGEST verification (the prune may lag the

delete arbitrarily; atomic-at-delete is a special case). everEdged is re-added only by GFold when

a future journal Add re-references the hash (resurrection), exactly as in the implementation.

$GForget(h) \triangleq$

$\wedge \neg present[h] \wedge h \in everEdged \wedge InDeg(h) = 0$

$\wedge everEdged' = everEdged \setminus \{h\}$

$\wedge$ UNCHANGED $\langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

Journal trim: INV_JOURNAL_COVERAGE — only below the durable folded cursor.

$Trim(s) \triangleq$

$\wedge trimBase[s] < cursor[s]$

$\wedge trimBase' = [trimBase \text{ EXCEPT } ![s] = @ + 1]$

$\wedge$ UNCHANGED $\langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, rootEdges, treeEdges, marker, everEdged, pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

— stage 4: heartbeats + debris + full-GC cut

$WHbStart(w) \triangleq$

$\wedge EnableDebris \wedge \neg hbAlive[w]$

$\wedge hbAlive' = [hbAlive \text{ EXCEPT } ![w] = \text{TRUE}]$

$\wedge$ UNCHANGED $\langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wDeps, wView, creator, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv \rangle$

$WHbRenew(w) \triangleq$
 $\wedge EnableDebris \wedge hbAlive[w] \wedge \neg wedged[w] \wedge hbSeq[w] < MaxRound + 2$
 $\wedge hbSeq' = [hbSeq \text{ EXCEPT } ![w] = @ + 1]$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase,$
 $roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker,$
 $everEdged, pendCasc, wDeps, wView, creator, hbAlive, wedged, hbObs, fgPhase,$
 $fgCut, fgRefs, fgSeen, reg, wEv \rangle$

$Wedge(w) \triangleq$ environment: renewal thread stuck; the writer can still publish
 $\wedge EnableDebris \wedge \neg wedged[w]$
 $\wedge wedged' = [wedged \text{ EXCEPT } ![w] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase,$
 $roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker,$
 $everEdged, pendCasc, wDeps, wView, creator, hbAlive, hbSeq, hbObs, fgPhase,$
 $fgCut, fgRefs, fgSeen, reg, wEv \rangle$

$WCrash(w) \triangleq$ process death: heartbeat gone, build state lost; uploads stay as debris
 $\wedge EnableDebris \wedge hbAlive[w]$
 $\wedge hbAlive' = [hbAlive \text{ EXCEPT } ![w] = \text{FALSE}]$
 $\wedge wDeps' = [wDeps \text{ EXCEPT } ![w] = \{\}]$
 $\wedge wEv' = [wEv \text{ EXCEPT } ![w] = \{\}]$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase,$
 $roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker,$
 $everEdged, pendCasc, wView, creator, hbSeq, wedged, hbObs, fgPhase, fgCut,$
 $fgRefs, fgSeen, reg \rangle$

$GObserveHb(w) \triangleq$ first read of the GC observation window
 $\wedge EnableDebris$
 $\wedge hbObs' = [hbObs \text{ EXCEPT } ![w] = \text{IF } hbAlive[w] \text{ THEN } hbSeq[w] \text{ ELSE } -1]$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase,$
 $roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker,$
 $everEdged, pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, fgPhase,$
 $fgCut, fgRefs, fgSeen, reg, wEv \rangle$

Debris classification: present, never journal-known, creator's heartbeat absent OR unchanged
since the earlier observation (the second read happens in this guard). Same retire tail.
 $GDebrisRetire(l, h) \triangleq$
 $\wedge EnableDebris \wedge gcPhase[l] = \text{"retiring"}$
 $\wedge present[h] \wedge h \notin everEdged \wedge creator[h] \in Writers$
 $\wedge \text{LET } c \triangleq creator[h] \text{ IN}$
 $\quad \vee \neg hbAlive[c]$
 $\quad \vee (hbObs[c] \geq 0 \wedge hbObs[c] = hbSeq[c])$
 $\wedge \neg \exists e \in retired : e.h = h \wedge e.t = tokOf[h]$
 $\wedge retired' = retired \cup \{[h \mapsto h, t \mapsto tokOf[h], r \mapsto roundOf[l]]\}$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, inflight, gcRound, gcPhase, roundOf,$
 $fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged,$

pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgPhase, fgCut, fgRefs, fgSeen, reg, wEv

—— full-GC walk: per-shard atomic reads recording the EXACT cut; commit CAS-guarded ——
 $FGRead(s) \triangleq$

$\wedge EnableDebris \wedge fgPhase \in \{\text{"idle"}, \text{"reading"}\} \wedge s \notin fgSeen$
 $\wedge fgCut' = [fgCut \text{ EXCEPT } ![s] = Len(man[s].log)]$
 $\wedge fgRefs' = [fgRefs \text{ EXCEPT } ![s] = man[s].refs]$
 $\wedge fgSeen' = fgSeen \cup \{s\}$
 $\wedge fgPhase' = \text{IF } fgSeen \cup \{s\} = Shards \text{ THEN "read" ELSE "reading"}$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, cursor, trimBase, rootEdges, treeEdges, marker, everEdged, pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, reg, wEv \rangle$

Rebuild authority EXACTLY through the cut: *refs* snapshot $\rightarrow$ root edges; reachable *trees* $\rightarrow$ expansion; cursors := the versions actually read. Claimed authority $\triangleq$ incorporated state.

The commit FAILS (walk aborts to idle) if any cursor already advanced past its cut.

SabotageCutOverclaim: cursors jump to CURRENT *log* length while edges come from the cut.

$FGCommit \triangleq$

$\wedge EnableDebris \wedge fgPhase = \text{"read"}$
 Under the sabotage, only commit when a publish actually landed in the read-commit gap — focuses *TLC* on the intended counterexample instead of vacuous overclaims.
 $\wedge (\neg SabotageCutOverclaim) \vee (\exists s \in Shards : Len(man[s].log) > fgCut[s])$
 $\wedge \text{IF } \forall s \in Shards : cursor[s] \leq fgCut[s]$
 THEN $\wedge rootEdges' = \text{UNION } \{\langle s, h \rangle : h \in fgRefs[s] : s \in Shards\}$
 $\wedge \text{LET } trees \triangleq \{h \in \text{UNION } \{fgRefs[s] : s \in Shards\} : h \in TreeHashes\}$
 $\wedge treeEdges' = \text{UNION } \{\langle t, c \rangle : c \in Children[t] : t \in trees\}$
 $\wedge marker' = trees$
 $\wedge everEdged' = \text{UNION } \{Reach(r) : r \in \text{UNION } \{fgRefs[s] : s \in Shards\}\}$
 $\wedge cursor' = \text{IF } SabotageCutOverclaim$
 THEN $[s \in Shards \mapsto Len(man[s].log)]$
 ELSE $[s \in Shards \mapsto fgCut[s]]$
 ELSE UNCHANGED $\langle rootEdges, treeEdges, marker, everEdged, cursor \rangle$
 $\wedge fgPhase' = \text{"idle"} \wedge fgSeen' = \{\}$
 $\wedge \text{UNCHANGED } \langle present, tokOf, nextTok, deadTok, man, retired, inflight, gcRound, gcPhase, roundOf, fencedSet, fencePos, trimBase, pendCasc, wDeps, wView, creator, hbAlive, hbSeq, wedged, hbObs, fgCut, fgRefs, reg, wEv \rangle$

—— next / spec ——
 $Next \triangleq$

$\vee \exists w \in Writers, h \in Hashes : WCreate(w, h) \vee WReuse(w, h)$
 $\vee \exists w \in Writers, h \in Hashes : WOverwrite(w, h)$
 $\vee \exists w \in Writers, h \in Hashes : WResurrect(w, h) \vee WEvidence(w, h) \vee WResolveEvidence(w, h)$
 $\vee \exists w \in Writers, h \in Hashes : WReobserve(w, h) \vee WEvObserve(w, h)$
 $\vee \exists w \in Writers : WRefreshView(w) \vee WAbandon(w)$

$\vee \exists s \in \text{Shards} : WRegister(s)$
 $\vee \exists w \in \text{Writers}, s \in \text{Shards}, h \in \text{Hashes} : WPublish(w, s, h)$
 $\vee \exists s \in \text{Shards}, h \in \text{Hashes} : WDrop(s, h)$
 $\vee \exists l \in \text{Leaders} : GStartRound(l) \vee GFenceRegistry(l) \vee GEndRound(l)$
 $\vee \exists s \in \text{Shards} : GFold(s) \vee Trim(s)$
 $\vee \exists l \in \text{Leaders}, h \in \text{Hashes} : GRetire(l, h)$
 $\vee \exists l \in \text{Leaders}, s \in \text{Shards} : GFenceShard(l, s)$
 $\vee \exists l \in \text{Leaders}, e \in \text{retired} : GRecheckDelete(l, e)$
 $\vee \exists d \in \text{inflight} : Land(d)$
 $\vee \exists t \in \text{TreeHashes} : ApplyPendCascade(t)$
 $\vee \exists w \in \text{Writers} : WHbStart(w) \vee WHbRenew(w) \vee Wedge(w) \vee WCrash(w) \vee GObserveHb(w)$
 $\vee \exists l \in \text{Leaders}, h \in \text{Hashes} : GDebrisRetire(l, h)$
 $\vee \exists h \in \text{Hashes} : GForget(h)$
 $\vee \exists s \in \text{Shards} : FGRead(s)$
 $\vee FGCommit$

$Spec \triangleq Init \wedge \square[Next]_{vars}$

$TypeOK \triangleq$

invariants

$\wedge present \in [Hashes \rightarrow \text{BOOLEAN}]$
 $\wedge tokOf \in [Hashes \rightarrow 0 \dots MaxToken]$
 $\wedge nextTok \in [Hashes \rightarrow 1 \dots MaxToken + 1]$
 $\wedge deadTok \in [Hashes \rightarrow \text{SUBSET Toks}]$
 $\wedge \forall s \in \text{Shards} : man[s].fence \in 0 \dots MaxRound \wedge man[s].refs \subseteq Hashes$
 $\quad \wedge man[s].log \in Seq(Rec)$
 $\wedge gcRound \in 0 \dots MaxRound$
 $\wedge creator \in [Hashes \rightarrow \text{Writers} \cup \{\text{"none"}\}]$
 $\wedge hbAlive \in [Writers \rightarrow \text{BOOLEAN}]$
 $\wedge hbSeq \in [Writers \rightarrow 0 \dots (MaxRound + 2)]$
 $\wedge wedged \in [Writers \rightarrow \text{BOOLEAN}]$
 $\wedge hbObs \in [Writers \rightarrow \{-1\} \cup (0 \dots (MaxRound + 2))]$
 $\wedge fgPhase \in \{\text{"idle"}, \text{"reading"}, \text{"read"}\}$
 $\wedge fgCut \in [Shards \rightarrow 0 \dots MaxLog]$
 $\wedge fgRefs \in [Shards \rightarrow \text{SUBSET Hashes}]$
 $\wedge fgSeen \subseteq Shards$
 $\wedge reg.ns \subseteq Shards$
 $\wedge reg.man \in [Shards \rightarrow \text{BOOLEAN}]$
 $\wedge reg.fence \in 0 \dots MaxRound$
 $\wedge reg.floor \in [Shards \rightarrow 0 \dots MaxRound]$
 $\wedge reg.univ \in [Leaders \rightarrow \text{SUBSET Shards}]$
 $\wedge reg.done \in [Leaders \rightarrow \text{BOOLEAN}]$
 $\wedge wEv \in [Writers \rightarrow \text{SUBSET } (Hashes \times (0 \dots MaxRound))]$

Model self-check (*EnableRegistry*): manifest ACTIVITY implies the manifest exists — every *refs* member or journal record was put there by a publish or a fence mint, both of which set *reg.man*.

(Under *SabotageNoFence* the fence skips the manifest entirely, so it also mints nothing.)

$$\begin{aligned} INV_MAN_EXISTS &\triangleq \forall s \in Shards : \\ &\quad (man[s].refs \neq \{\} \vee Len(man[s].log) > 0) \Rightarrow reg.man[s] \end{aligned}$$

Roots hold LOGICAL hashes only; the publish-time token dependency is transient by design (the real *publish_dependency_set* is writer-local). Hence *NO_DANGLE/NO_LOSS* are about logical presence — any current incarnation satisfies a reader — exactly the spec’s reader contract.

$$\begin{aligned} INV_NO_DANGLE &\triangleq \forall s \in Shards : \forall h \in man[s].refs : present[h] \\ INV_NO_LOSS &\triangleq \forall h \in ReachableSet : present[h] \end{aligned}$$

INV_NO_RETURN models the LATE-DELETE-SAFETY facet of the spec’s invariant (a deleted token is never current again). The publish-gate facet (“a condemned token is never a valid dependency of a publish”) is enforced by the W-PUBLISH-GATE guard and proven load-bearing by the *sab_noretireview* counterexample — it is NOT a state invariant (an unpublished writer may hold a dependency on a token that *GC* legitimately deletes; the gate catches it at publish).

$$INV_NO_RETURN \triangleq \forall h \in Hashes : present[h] \Rightarrow tokOf[h] \notin deadTok[h]$$

This is the ORDERING half only of the spec’s *INV_JOURNAL_COVERAGE*: trim never advances past the durable folded cursor. The coverage half (the journal retains every folded-but-untrimmed record) holds BY CONSTRUCTION in this model — *Trim* only advances a watermark below cursor and never pops journal records — so this invariant cannot, by itself, catch a coverage bug.

$$INV_JOURNAL_COVERAGE \triangleq \forall s \in Shards : trimBase[s] \leq cursor[s]$$

Monotonicity of *GC* state — checked as an action property (PROPERTY in configs).

NOTE on form: $\Box[A]_{vars}$ is the standard *TLC* action property; the *_vars* subscript exempts stuttering steps, so this does NOT over-constrain. The *Len(log)*-monotone clause is valid only because *Trim* advances *trimBase* without physically shrinking the *log* — if real *log* compaction is ever modeled, revise this clause together with the *Trim* action.

$$\begin{aligned} MonotoneGC &\triangleq \Box[\wedge gcRound' \geq gcRound \\ &\quad \wedge \forall s \in Shards : \wedge cursor'[s] \geq cursor[s] \\ &\quad \wedge trimBase'[s] \geq trimBase[s] \\ &\quad \wedge man'[s].fence \geq man[s].fence \\ &\quad \wedge Len(man'[s].log) \geq Len(man[s].log)]_{vars} \end{aligned}$$

Registry monotonicity (*B91* review follow-up; checked as PROPERTY in registry-enabled configs):

the registered set is append-only, the registry fence never lowers, an existing manifest never becomes “absent”, and a namespace’s registration floor is immutable once written (it is set exactly once, by *WRegister*, when the namespace ENTERS *reg.ns*). *reg.univ* and *reg.done* are per-round work state and deliberately NOT monotone.

$$\begin{aligned} MonotoneRegistry &\triangleq \Box[\wedge reg.ns \subseteq reg'.ns \\ &\quad \wedge reg'.fence \geq reg.fence \\ &\quad \wedge \forall s \in Shards : \wedge (reg.man[s] \Rightarrow reg'.man[s]) \\ &\quad \wedge (s \in reg.ns \Rightarrow reg'.floor[s] = reg.floor[s])]_{vars} \end{aligned}$$

StateConstraint $\triangleq$

$$\begin{aligned} &\wedge \forall s \in Shards : Len(man[s].log) \leq MaxLog \\ &\wedge Cardinality(inflight) \leq 2 \\ &\wedge \forall w \in Writers : Cardinality(wDeps[w]) \leq 3 \end{aligned}$$

$$\wedge \forall w \in \text{Writers} : \text{Cardinality}(\text{wEv}[w]) \leq 2$$

Liveness (checked only on the small stage – 2 bounds): a permanently-unreachable journal-known object is eventually deleted. Fairness on the *GC* pipeline + delete landing.

$$\begin{aligned} \text{FairSpec} &\triangleq \text{Spec} \wedge \text{WF}_{\text{vars}}(\exists s \in \text{Shards} : \text{GFold}(s)) \\ &\quad \wedge \text{WF}_{\text{vars}}(\exists l \in \text{Leaders} : \text{GStartRound}(l)) \\ &\quad \wedge \text{WF}_{\text{vars}}(\exists l \in \text{Leaders}, h \in \text{Hashes} : \text{GRetire}(l, h)) \\ &\quad \wedge \text{WF}_{\text{vars}}(\exists l \in \text{Leaders}, s \in \text{Shards} : \text{GFenceShard}(l, s)) \\ &\quad \wedge \text{WF}_{\text{vars}}(\exists l \in \text{Leaders}, e \in \text{retired} : \text{GRecheckDelete}(l, e)) \\ &\quad \wedge \text{WF}_{\text{vars}}(\exists l \in \text{Leaders} : \text{GEndRound}(l)) \\ &\quad \wedge \text{WF}_{\text{vars}}(\exists d \in \text{inflight} : \text{Land}(d)) \end{aligned}$$

$$\begin{aligned} \text{NoLeakForever} &\triangleq \\ &\quad \forall h \in \text{Hashes} : \\ &\quad \quad \square((\text{present}[h] \wedge h \in \text{everEdged} \wedge h \notin \text{ReachableSet}) \\ &\quad \quad \Rightarrow \Diamond(\neg \text{present}[h] \vee h \in \text{ReachableSet})) \end{aligned}$$